



Setup

In this chapter, you will learn:

- To set up your local development environment.
- The outline of the Odoo database structure.
- To export and import an Odoo database in your local environment.
- To have an Odoo instance up and running.

Install

There are multiple ways to [install Odoo](#), depending on the intended use case. This documentation assumes you use the [source install](#) (running Odoo from the source code), which is best suited for Odoo designers and developers.

Databases

Structure

Every Odoo application works similarly; they are built with the same logic. A model contains fields and relational fields that link to other models. Each model has views representing all its fields, with backend and frontend views.

Models

The basis of Odoo is models. Models use fields to record the data. Records are stored in a database: they are therefore linked to a model. In Odoo, you can find the different models in the backend by enabling the [developer mode](#) and then going to **Settings ▸ Technical ▸ Database Structure: Models**.

Model Description ?	Page	Type ?	Base Object
Model ?	website.page	In Apps ?	website, website_sale
Order ?	website_id		
Abstract Model ?	☐		
Transient Model ?	☐		
Has Mail Thread ?	☐		
Has Mail Activity ?	☐		
Has Mail Blacklist ?	☐		

Fields

Access Rights

Record Rules

Notes

Views

Website Forms

1-40 / 64 < >

Field Name	Field Label	Field Type	Required	Readonly	Indexed	Type
active	Active	boolean	☐	☐	☐	Base Field 
arch	View Architecture	text	☐	☐	☐	Base Field 

Fields

In a model, we will centralize fields (field names we need to target in our code).

See also

[Fields and widgets](#)

Classic fields

- Date
- Char
- Selection
- ...

Relational fields

Relational fields call a field from another model. They allow you to link models together and make them interact easily. In other words, when you use a relational field, you link a record with another one (located on another model), enabling you to retrieve the content of the fields located on this linked record.

- **Many2one** fields are filled in by choosing one record from a list of records on another model (from *many* records, you select *one*). For example, the *customer* field on a

could list on a contact all their existing quotations (from *one* record, you display *many*).

- **Many2many** fields are filled in by choosing one or several records from a list of records on another model. For example, you can put several tags on one product, and several products can use the same tags (from *many* records, you can select *many*).

Views

Views define how records should be displayed to end-users. They are specified in XML, meaning they can be edited independently from the models they represent. They are flexible and allow deep customization of the screens they control.

Backend vs. Frontend

- **Backend views:** Kanban, List, Form, etc.
- **Frontend view:** QWeb

Static vs. Dynamic

- **Static pages** have stable content, such as the homepage. You can define their URL and set some properties like published, indexed, etc.
- **Dynamic pages** are dynamically generated, such as the product page. Their URL is dynamic and is accessible to all by default (this can be changed by configuring access rights).

Base vs. Inherited vs. Duplicated

- **Base view:** View natively implemented by Odoo. It is directly derived from their model. A base view should never be altered as it allows updating an Odoo database without overwriting a client's modification.

Modifications always take place in either a duplicated or inherited view with some differences:

- **Inherited view** always has an `inherit_id` and applies modifications based on the source code of the base view it inherits from.
- **Duplicated view** is a duplication from another one. Most of the time this kind of view is created by Odoo when the end-user applies modifications through the Website Builder. This mechanism prevents data loss when the module is updated because only the source view will be updated, not the duplicated one. If a duplicated view exists, both the original view and the duplicated one are visible in the list but only the duplicated one has an `external_id` (an `external_id` set means that the view has been created by the source code of a module).

You can directly go to the [Theming](#) chapter if you do not need to import an existing database.

Dump

Odoo SaaS

1. Log in to a user account with sufficient access rights.
2. Go to `<database_url>/saas_worker/dump` .

Odoo.sh

1. Connect to Odoo.sh.
2. Select the branch you want to back up.
3. Choose the **BACKUPS** tab.
4. Click the **Create Backup** button.
5. When the process is over, a notification appears. Open it and click the **Go to Backup** button.
6. Click the **Download** icon. Select **Testing** under **Purpose** and **With filestore** under **Filestore**.

Making a dump can take a while, depending on the build's size.

Purpose ☒ **Testing**
☐ **Exact dump**

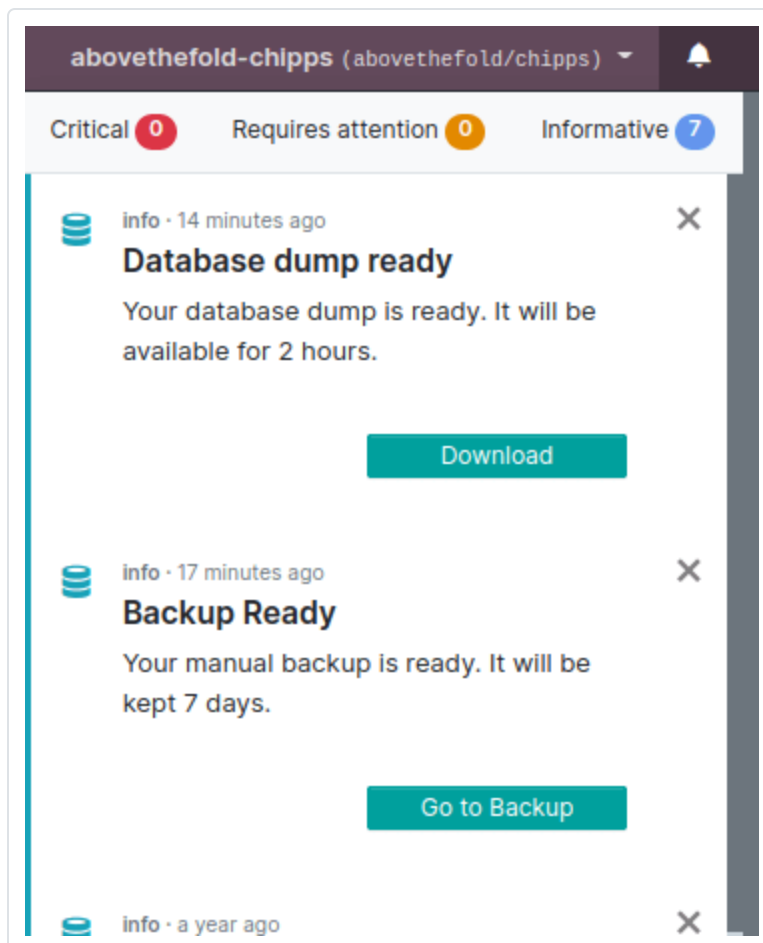
Filestore ☐ **Without filestore**
☒ **With filestore**

The filestore makes the dump quite larger. It contains all the files and attachments but is often not necessary.

You will be notified when the backup is ready for download.

[Discard](#)[Start](#)

7. You will receive a notification when the dump is ready to be downloaded. Open it and click on **Download** to get your dump.



- macOS: /Users/<User>/Library/Application Support/Odoo/filestore/<database_name>
- Linux: /home/<User>/.local/share/Odoo/filestore/<database_name>

Note

`/Library` is a hidden folder.

Database setup

Create an empty database.

```
createdb <database_name>
```

Import the SQL file in the database that you just created.

```
psql <database_name> < dump.sql
```

Reset the admin user password.

```
psql \c  
<database_name>  
update res_users set login='admin', password='admin' where id=2;
```

If necessary, disable the two-factor authentication enforcing policy option.

```
psql <database-name>  
update res_users set totp_secret='' where id=2;
```

Getting started

Running Odoo

Once all dependencies are set up, Odoo can be launched by running `odoo-bin`, the command-line interface of the server. It is located at the root of the Odoo Community

To configure the server, you can specify command-line arguments or a configuration file. The first method is presented below.

The **CLI** offers several functionalities related to Odoo. You can use it to **run the server**, scaffold an Odoo theme, populate a database, or count the number of lines of code.

Shell script

A typical way to **run the server** would be to add all command line arguments to a `.sh` script.

Example

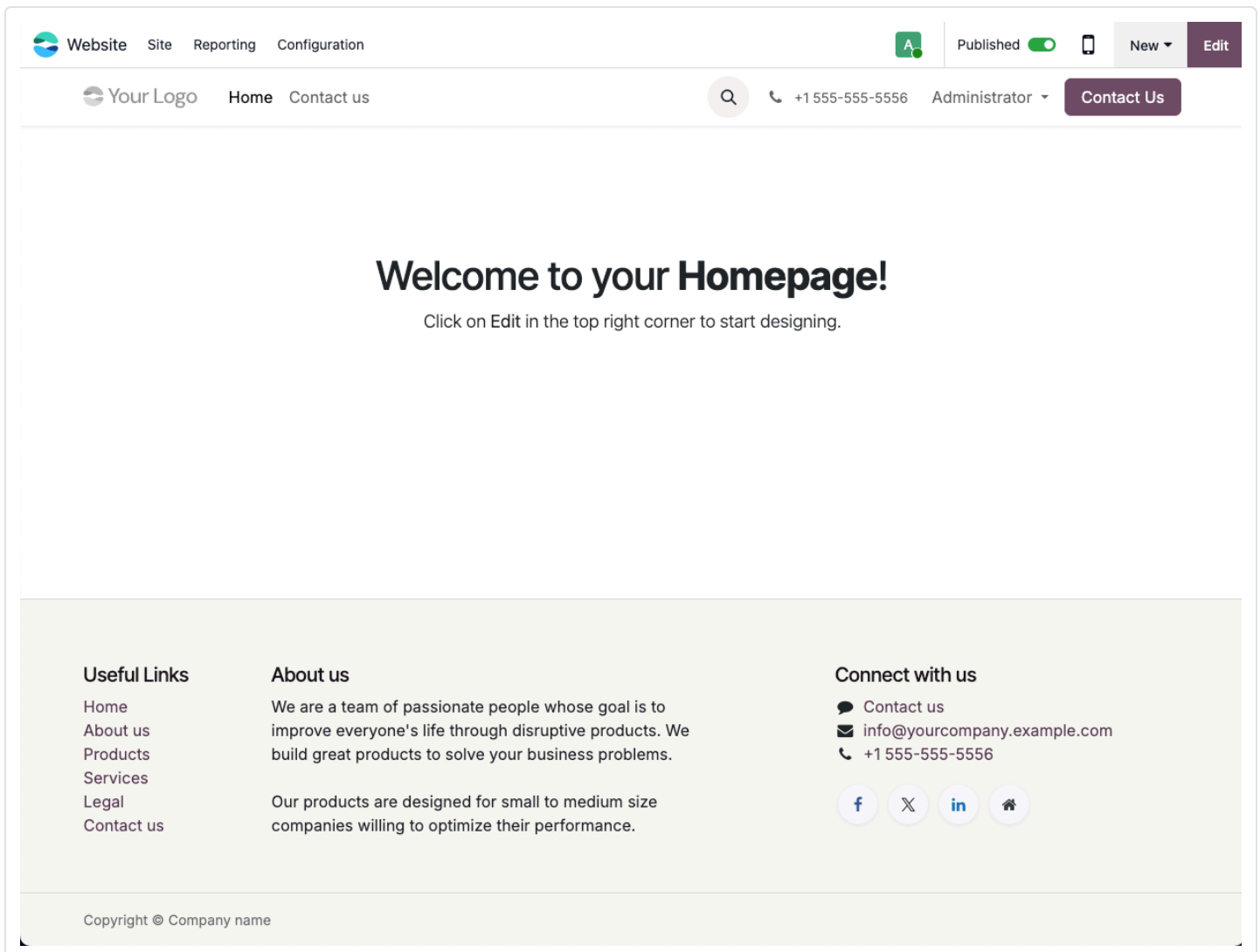
```
./odoo-bin --addons-path=../enterprise,addons --db-filter=<database> -d <databa
```

Folder	Description
--addons-path	Comma-separated list of directories in which modules are stored. These directories are scanned for modules.
-d	database(s) used when installing or updating modules.
--database	
--db-filter	Hides databases that do not match the filter.
-i	Comma-separated list of modules to install before running the server. (requires -d)
--init	
-u	Comma-separated list of modules to update before running the server. (requires -d)
--update	
--reinit	A comma-separated list of modules to reinitialize before starting the server. (requires -d)
--dev	Comma-separated list of features. For development purposes only. More info

Sign in

After the server has started (the INFO log `odoo.modules.loading: Modules loaded.` is printed), open <http://localhost:8069> in your web browser and log in with the base administrator account.

Type **admin** for the email and **admin** for the password.



Tip

Hit **CTRL+C** to stop the server. Do it twice if needed.

Developer mode

See also

[Developer mode \(debug mode\)](#)

Get Help

[Contact Support](#)

[Ask the Odoo Community](#)

